

Wrench Documentation - v0.1.1

You can access the project's Github Page [here](#).

Tutorial

This episode teaches the basic workings and the prototype structure of the language.

You can check the [Language Reference](#) episode for the definitions of the syntax rules, and [Wrench Standard Library](#) for the standard library.

1. Hello, World!

Well, hello. Welcome to Wrench! Unlike a lot of languages you're familiar with that run inside Virtual Machines (or for some, require massive background runtimes), Wrench tries to get you as close to the hardware as possible without forcing you to write raw assembly by hand. It is a native language, which means its code compiles directly into binary machine code.

Let us begin with your very first program and learn how the compiler is used.

1.1 Your First Program

Create a file named `hello.wr`. In Wrench, every statement must end with a semicolon (`;`).

```
print("Hello, World!");
```

That's it! No need for a `main()` function wrapper or importing massive standard I/O libraries just to print text. Wrench handles it at the kernel level.

Added in version 0.1.1: Added support for formatted printing via `printf`. You can interpolate variables directly inside format strings using curly braces `{}` or placeholder specifiers.

```
var name = "World";  
printf("Hello, {name}!\n");
```

1.2 The Command Line Interface (CLI) and Flags

The compiler is invoked via the command line. To compile your program, you simply pass the `.wr` source code to be compiled as an input argument:

```
python wrench.py hello.wr
```

By default, this will compile the code and generate a native executable named `hello`.

However, the CLI provides several arguments and flags to control the compilation pipeline:

- `-o` or `-output`: Sets the executable file name to be produced. If you do not provide this, it defaults to the original file's name.
- `-ns` or `-no-strict`: Turns off strict mode in semantic analyzation. By default, Wrench enforces strict type-checking to keep your memory safe. You can use this flag to bypass those rules during experimental prototyping.
- `-k` or `-keep`: Keeps the temporary files created during compilation (`.asm` / `.o`). Normally, Wrench cleans up these intermediate files. Use this flag if you want to inspect the actual x86-64 assembly code generated by the compiler.
- `-r` or `-run`: Automatically run the program and delete the executable. This is useful for testing code on the fly without cluttering your directory with binaries.

Example: Rapid Prototyping If you want to quickly test your `hello.wr` script and immediately see the output, just use the run flag:

```
python wrench.py hello.wr -r
```

2. Variables, Types and Tolerances

Wrench provides a robust static type system, but it does not force you into endless boilerplate. It combines low-level memory safety with high-level developer ergonomics.

2.1. Primitive Types & Inference

Wrench supports standard primitive types that map directly to hardware registers. You can declare variables explicitly using `int`, `double`, `str`, `char`, and `bool`.

Changed in version 0.1.1: The null/empty evaluation type previously referred to as `void` is now consistently named `unit`.

```
int health = 100;
double pi = 3.14;
str greeting = "Welcome to Wrench";
bool is_active = true;
```

If you don't want to type out the explicit data type, you can use the `var` keyword. The Wrench compiler will infer the type based on the assigned value during compilation. Once inferred, the type is statically locked.

```
var score = 50;           // Inferred as int
var username = "Root";   // Inferred as str
```

2.2. Ergonomics, Case Sensivity & Operator Tolerance

Wrench introduces several unique quality-of-life features to make writing code easier, faster and less error-prone.

Hybrid Case Sensivity: Wrench has a unique way in handling letter casing. All built-in language keywords (like `if`, `while`, `define`, `int`) are **case insensitive**. The Lexer automatically normalizes them, meaning `DeFiNe` and `define` are treated exactly the same. However, your custom identifiers (variable names, function names, and classes) remain **case-sensitive**.

Operator Tolerance:

In many languages, accidentally typing `=+` instead of `+=` throws a syntax error that's annoying as hell. In Wrench, the Lexer is tolerant of directional typos for compound assignments.

```
var x = 10;
x += 5; // Perfectly valid, identical to x += 5;
x -= 2; // Identical to x -= 2;
```

Furthermore, to eliminate the notorious `=` (assignment) vs `==` (comparison) bugs found in C-like languages, Wrench uses a distinct equality operator: `=?`. `?=` is also perfectly valid.

```
if (x =? 13) {
    print("x is exactly 13"); // No sh sherlock
}
```

Reminder: Never forget your semicolons (`;`)! It's required for all assignments, returns, and function calls.

2.3. Deep Dive: Type Casting

Because Wrench operates in strict mode by default, you cannot implicitly mix incompatible types. Instead, you use the `cast` keyword. The `cast` operator is incredibly powerful and serves three distinct purposes in Wrench:

1. Primitive Conversions: Used to convert standard data types. For example, converting an integer into a string for terminal output, or safely flooring a double to an integer.

```
var age = 25;
var text = age cast str; // Dynamically allocates a string representation of 25
```

2. Object Hierarchy (Upcasting & Downcasting): When working with Object-Oriented structures, `cast` allows you to navigate inheritance trees. While assigning a child object to a parent variable (Upcasting) is handled implicitly by the compiler, bringing a parent reference back to a child type (Downcasting) requires an explicit cast.

```
Character enemy = new Goblin(); // Implicit Upcast
var goblin_ref = enemy cast Goblin; // Explicit Downcast
```

3. Raw Memory Reinterpretation: For low-level systems programming, `cast` is used to forcefully interpret raw memory addresses (like `hex` or `ptr` types) into usable data types or class structures, similar to how `reinterpret_cast` works in C++. This is essential when reading incoming data from network sockets, hardware buffers, or custom memory allocators.

3. Control Flow and Loops

Wrench takes the flow concepts that you're probably already familiar with and streamlines them, eliminating common sources of bugs like switch fall-throughs and ambiguous keywords.

3.1. Conditions & Branching

Wrench uses standard `if` and `else` blocks, but it introduces `butif` as a replacement for `else if` or `elif`.

```
var status = 200;

if (status ==? 200) {
    print("OK");
} butif (status ==? 404) {
    print("Not Found");
} else {
    print("Unknown Error");
}
```

The Switch Statement (No Fall-Through): In languages like C or C++, forgetting a `break;` statement inside a `switch` block causes the code to “fall through” and accidentally execute the next case. This is a notorious source of bugs. Wrench completely eliminates this risk. Every `case` block automatically breaks at the end. You never have to write `break;` manually.

```
var opcode = 1;

switch (opcode) {
    case 0:
        print("Halt");
    case 1:
        print("Add");
    default:
        print("Unknown Opcode");
}
```

3.2. Foreach Logic and Loops

Wrench provides a standard `while` loop that runs as long as its condition evaluates to true.

```
var count = 0;
while (count < 3) {
    print_int(count);
    count += 1;
}
```

For iterating over collections, Wrench uses a `for ... in ...` syntax. This acts as a “foreach” loop, automatically extracting each element from an array without needing manual index tracking or length calculations.

```
var weapons = ["Sword", "Bow", "Axe"];

for weapon in weapons {
    print(weapon);
}
```

3.3. The 'limits' Function and Loop Control

When you need to loop over a specific numerical range, you use Wrench's built-in

`limits(start, end)` function.

Important Difference: If you are coming from Python, you might expect `range(1, 5)` to generate numbers from 1 up to 4, stopping before the final number. Wrench's `limits` function is **inclusive** of the target number. Calling `limits(1, 5)` will generate exactly `1, 2, 3, 4, 5` (By the way Python rules, even though i don't think it's much, i highly think it's slightly underestimated in the world of coding).

```
for i in limits(1, 5) {  
    print_int(i); // Prints 1, 2, 3, 4, 5  
}
```

Loop Control (break & goon): Wrench uses standard loop control mechanisms but renames one keyword.

- `break;` : Immediately exits and terminates the nearest enclosing loop.
- `goon;` : (Short for "Go On" -i know what it sounds like-). This replaces the traditional `continue` keyword. It skips the rest of the current iteration and forces the loop to proceed to the next element.

```
for i in limits(1, 10) {  
    if (i == 5) {  
        break; // Stops the loop entirely when i is 5  
    } butif (i % 2 == 0) {  
        goon; // Skips the print statement for even numbers  
    }  
  
    print_int(i); // Will only print 1, 3  
}
```

4. Functions

Functions in Wrench are straightforward. They map directly to hardware-level jumps and strictly follow the kernel calling conventions (Current version only supports System V ABI conventions, which are used by Unix-like operating systems like the Linux kernel. If you want to learn more about it, you can check the *OS Dev Wiki* [here](https://wiki.osdev.org/System_V_ABI) <https://wiki.osdev.org/System_V_ABI>).

4.1. Basic Functions

To create a standard function, use the `define` keyword followed by the function name and its parameter list. Wrench uses static typing for parameters, but you do not need to explicitly declare the return type in the function signature—the compiler infers it dynamically based on what you return.

Added in version 0.1.1: Functions now support explicit return type syntax using the arrow operator `-> ReturnType`.

```
define greet_user(str name) -> unit {  
    print("Welcome back, " + name);  
}  
  
greet_user("Admin");
```

To return a value from a function, use the `return` keyword. If you just want to exit a function early without returning a specific value, you can simply write `return;`.

```
define calculate_area(int width, int height) {  
    if (width <= 0 or height <= 0) {  
        return 0; // Early exit for invalid dimensions  
    }  
  
    var area = width * height;  
    return area;  
}  
  
var room_area = calculate_area(5, 4);
```

4.2. Anonymous Functions (Anfuncs)

Sometimes you need a function for a quick, one-off operation, or you want to pass a function around as a variable. Wrench supports anonymous functions via the `anfunc` keyword.

An `anfunc` does not have a name. Instead, you assign it directly to a variable. This makes functions “first-class citizens” in Wrench, allowing you to treat executable code blocks just like regular data pointers.

```
// Assigning an anonymous function to a variable
var multiply = anfunc(int a, int b) {
    return a * b;
};

// Calling it dynamically
var result = multiply(10, 5);
print_int(result); // Prints 50
```

Note: Since assigning an anonymous function to a variable is a variable assignment statement, do not forget to put a semicolon (`;`) at the very end of the closing brace `};`.

5. Arrays and Deep Memory Management

Wrench gives you the raw power of memory management (which i strongly support, in a world where every possibility and right is being stripped away, including the internet and the computing), but takes some safety measures where it actually matters.

5.1. Arrays and Safety

All you have to do to crate an array is to use two square brackets:

```
var weapons = ["Sword", "Bow", "Axe"];
print(weapons[1]); // Prints "Bow"
```

The Hidden Header Architecture: When you create an array (or allocate memory), Wrench automatically requests an extra 8 bytes of memory and places them invisibly right before your actual data. This “hidden header” permanently stores the length or size of your array in RAM.

Out-of-Bounds Protection: In older low-level languages, trying to read `weapons[10]` would silently read garbage memory or cause a Segmentation Fault. Wrench, checks the hidden header first automatically every array access. If the index is invalid, the compiler halts the operation and throws an Error (Code 99). This prevents crashes and allows you to handle the mistake.

5.2. Manual Memory Management

Wrench **does not** have a Garbage Collector (GC). Garbage collectors consume unpredictable CPU cycles and memory, which is not very compatible with low-level programming (Although, there will be an optional Garbage Collector developed in the later versions).

You are fully in charge. What you create, you must destroy. Or accept that at some point the program is going to crash.

Raw Pointers (ptr and address): When dealing with custom memory buffers or hardware addresses, you often work with raw pointers. In Wrench, you can explicitly type variables as `ptr`, `address`, or `hex`. A pointer is simply a numeric variable that holds the physical RAM address of your data, rather than the data itself.

5.3. Allocation and Cleanup (alloc & delete)

When you need a custom block of memory (like a dynamic text buffer or a network packet payload), you ask the Operating System for it using the built-in `alloc()` function.

`alloc()` talks directly to the kernel using Syscall 9 `mmap` to reserve raw memory (Future versions will include Win64 ABI, AArch64 macOS ABI and x86-64 macOS ABI. For details, see the [Roadmap](#).).

```
// Ask the OS for 1024 bytes of raw memory
var buffer = alloc(1024);
```

The Danger of Memory Leaks: Because there is no Garbage Collector, the OS assumes you are still using that memory until you explicitly give it back. If your function ends and you forget to return the memory, it becomes a “Memory Leak”—lost space that cannot be used again until your program completely shuts down (Which will probably be thing that causes the crashes i mentioned earlier).

The ‘delete’ Keyword: To safely return memory to the OS and prevent leaks, you must use the `delete` keyword. This instructs to read the hidden header of your pointer and triggers Syscall 11 (`munmap`) to instantly free the exact block of memory (Future versions will include Win64 ABI, AArch64 macOS ABI and x86-64 macOS ABI. For details, see the [Roadmap](#).).

```
define process_data() {
    var buffer = alloc(1024);

    // ... do heavy work with the buffer ...

    delete buffer; // Safely return memory to the OS!
}
```

6. Object-Oriented Basics

While Wrench is a low-level systems language, it provides an Object-Oriented Programming model to help you structure complex data easily. Objects are just contiguous blocks of memory, but Wrench makes them act high-level at the user end.

6.1. Classes and Constructors

You define an object's blueprint using the `class` keyword. Inside a class, you can declare variables (fields) and functions (methods).

To initialize an object when it is created, you define a special method called `init`. Inside any method, you can use the `this` keyword to access the specific instance of the object.

```
class Player {
    int health;
    str name;

    // The constructor
    define init(str player_name) {
        this.name = player_name;
        this.health = 100;
    }

    define take_damage(int amount) {
        this.health -= amount;
        print(this.name + " took damage!");
    }
}
```

6.2. Instantiation and Inheritance

To create an actual object from your class, you use the `new` keyword. This automatically calculates the size of the class, allocates the exact amount of memory needed via the kernel, and calls your `init` method.

Wrench also supports single inheritance using the `extends` keyword. A child class automatically inherits all the memory layout (fields) and methods of its parent.

```

class Boss extends Player {
    int armor;

    define init(str boss_name) {
        this.name = boss_name;
        this.health = 500;
        this.armor = 50;
    }
}

// Instantiating objects using 'new'
var hero = new Player("Arthur");
var dragon = new Boss("Fire Drake");

hero.take_damage(20);

```

Again, it apparently has no Garbage Collector, remember that objects created with `new` live in dynamically allocated memory. If you are destroying an object permanently, you should pass it to `delete` to avoid memory leaks.

7. Safety and Concurrency

Wrench handles errors and concurrency directly at the kernel level, bypassing any runtime.

7.1. Preventing Crashes (attempt, exclude, final)

Wrench traps fatal hardware errors, such as Segmentation Faults, using OS signal handlers. Instead of crashing, it routes the execution to safe blocks using three keywords:

- ```attempt```: The code block you want to run safely.
- ```exclude```: Executes if a crash occurs. The compiler automatically injects an `err` variable containing the error code here.
- ```final```: Always executes, regardless of errors.

```

attempt {
    var arr = limits(0, 5);
    print(arr[10]); // Throws an Out-of-Bounds error
} exclude {
    print("Error caught! Code: ");
    print_int(err); // 'err' is automatically available
} final {
    print("Cleanup complete;");
}

```

7.2. Background Tasks (async and when)

Wrench achieves concurrency through real multi-processing (forking) rather than user-space threads.

Async Functions: Prefixing a function with `async` tells the compiler to run it in a completely separate OS process. The main program continues without blocking.

```
async define background_task() {  
    // Heavy work runs in parallel  
}  
background_task();
```

Polling with 'when': The `when` keyword forks a lightweight background process that sleeps and continuously polls a condition. When the condition becomes true, it executes its block and terminates.

```
var system_ready = false;  
  
when (system_ready ==? true) {  
    print("System is ready!");  
}  
  
// ... other code ...  
system_ready = true; // The background observer detects this and fires.
```

8. Modularity and External Libraries

Wrench allows you to split your project into manageable pieces and seamlessly integrate with other low-level languages.

8.1. Wrench Modules (include)

You can import other Wrench (`.wr`) files using the `include` keyword. The compiler automatically resolves these files and merges their Abstract Syntax Trees before generating machine code.

```
include "math_utils"; // Automatically looks for math_utils.wr
```

To keep your global scope clean, you can import specific functions, classes, or variables using the `from ... include` syntax. You can also rename imported items using the `as` keyword to prevent naming collisions.

```
from "math_utils" include calculate_area as get_area;
```

8.2. Connecting to the Outside World (link & extern)

Wrench can natively interoperate with pre-compiled binaries written in C, C++, or raw Assembly.

Use the `link` keyword to instruct the compiler to pass an external object file (`.o`) to the GNU Linker (`ld`) during the final build stage.

```
link "physics_engine.o";
```

Once the external file is linked, you must declare its functions in Wrench using the `extern` modifier. This tells the compiler that the function body exists outside of your Wrench code.

```
// Declare the external function
extern define apply_gravity(int mass);

// Call it natively in Wrench
var result = apply_gravity(100);
```

Language Reference

Wrench Language Reference defines the grammar, types, memory management and how the system works.

You might as well check [Tutorial](#) if you're new and starting from scratch.

1. Design Philosophy & Architecture

Wrench is designed to bridge the gap between the ergonomic, developer-friendly syntax of high-level languages and the raw, bare-metal performance of low-level system languages. It enforces structure through explicit scoping brackets (`{}`, `[]`, `()`) rather than strict whitespace indentation.

1.1. Freestanding Execution & AOT Compilation

Wrench is an Ahead-Of-Time (AOT) compiled language. It does not rely on Virtual Machines (VM), Just-In-Time (JIT) compilers, or a heavy runtime environment.

The compiler translates Wrench source code directly into x86-64 Assembly (currently targeting the System V ABI). Because it does not require a background C library (libc) to function, Wrench is fundamentally freestanding. It interacts directly with the operating system kernel via native system calls (syscalls).

1.2 Memory Management

To maintain deterministic performance and a zero-overhead runtime, Wrench does not implement a Garbage Collector (GC).

- Memory allocation is handled transparently by the compiler for standard stack variables.
- Heap allocations (e.g., dynamic arrays, object instances created with `new`) rely on direct kernel memory mapping (e.g., `mmap`).
- Developers retain full control over the memory lifecycle using explicit keywords like `delete` and `alloc`, and can manipulate raw hardware addresses directly using primitive types such as `ptr`, `address`, and `hex`.

1.3. Object-Oriented Bare Metal

While Wrench utilizes an Object-Oriented Programming (OOP) paradigm, primitive data types (`int`, `char`, `double`) remain strictly lightweight. They do not carry the runtime overhead of a class instance unless explicitly cast or wrapped. Advanced mechanisms, such as signal-based exception handling (SIGSEGV catching), operate directly at the assembly level without requiring stack-unwinding libraries.

.._reference-lexical:

2. Lexical Structure

This section covers the basic lexical elements of Wrench, including comments, identifiers, keywords, and operators.

2.1. Comments

Wrench supports both single-line and multi-line comments. Comments are completely ignored by the lexer.

- **Single-line:** Starts with `//` and continues to the end of the line.
- **Multi-line:** Enclosed between `/*` and `*/`.

```
// This is a single-line comment
/* This is a
   multi-line comment */
```

2.2. Identifiers

Identifiers are used to name variables, functions, classes, and synchronization units. An identifier must start with a letter (A-Z, a-z) or an underscore (`_`), followed by any number of letters, digits, or underscores. Identifiers are case-sensitive.

Changed in version 0.1.1: Strict identifier validation (`expect_identifier`) is now enforced during parsing. Identifiers cannot share names with reserved keywords or built-in functions (e.g., `print`, `printf`, `len`, `alloc`, `realloc`, `free`, `read`, `get_mem32`, `syscall`).

2.3. Keywords

The following identifiers are reserved as keywords and cannot be used as regular variable or function names:

- **Control Flow:** `if`, `else`, `butif`, `switch`, `case`, `default`, `for`, `while`, `break`, `goon`
- **Data Types:** `int`, `integer`, `double`, `char`, `str`, `string`, `bool`, `var`, `unit`, `hex`, `ptr`, `pointer`, `address`
- **Logic & Operators:** `and`, `or`, `not`, `nand`, `nor`, `xor`, `xnor`, `in`, `is`, `same`, `cast`
- **OOP & Structure:** `class`, `new`, `this`, `extends`, `public`, `private`
- **Functions & Modules:** `define`, `anfunc`, `return`, `include`, `from`, `as`, `link`, `extern`
- **Memory & Error:** `delete`, `attempt`, `exclude`, `final`, `trigger`, `allege`
- **Concurrency & Events:** `async`, `sync`, `await`, `wait`, `with`, `when`, `global`
- **Constants:** `true`, `false`, `null`, `limits`

2.4. Operators & Operator Tolerance

Wrench provides standard mathematical and bitwise operators, but introduces a unique **operator tolerance** rule to improve developer ergonomics and prevent common syntax errors.

Added in version 0.1.1: Added the Arrow token `->` for explicit function and method return type declarations.

- **Assignment Tolerance:** Compound assignment operators can be written in either direction. For example, `+=` and `=+` are treated identically by the lexer. This applies to all compound assignments (`-=` / `=-`, `*=` / `=*`, `/=` / `=/`, etc.).
- **Relational Tolerance:** The equality check is written as `=?` or `?=`, intentionally avoiding the common `==` vs `=` confusion found in C-like languages. Other relational operators also feature directional tolerance (`>=` / `=>`, `!=` / `!=`, `<=` / `<=`).

```
x += 5; // Identical to x += 5;
if (x =? 10) { ... } // Identical to if (x ?= 10)
```

2.5. Punctuation

Wrench uses `{ }` for code blocks, `[ ]` for indexing and array literals, and `( )` for parameters and logical grouping.

Important: Wrench enforces a strict semicolon (`;`) rule. Every variable assignment, reassignment, return, delete, link, include, and trigger statement must explicitly end with a semicolon to successfully parse.

3. Types and Variables

Wrench offers a robust type system that balances the safety of static typing with the flexibility needed for low-level memory manipulation.

3.1. Primitive Types

Primitive types in Wrench are lightweight and translate directly into their hardware-level equivalents without background object overhead.

- **Integer:** `int` or `integer` (64-bit signed integer).
- **Floating-Point:** `double` or `float` (64-bit precision floating-point numbers mapped to xmm registers).
- **Character:** `char` (1-byte ASCII character).
- **Boolean:** `bool` (Evaluates to `1` for `true` and `0` for `false`).
- **String:** `str` or `string` (A contiguous array of characters ending with a null terminator).
- **Unit:** `unit` (Represents expressions or functions that return no value).

Changed in version 0.1.1: The non-value return evaluation type previously referred to as `void` has been renamed to `unit`.

3.2. Memory & Hardware Types

To support freestanding execution and bare-metal programming, Wrench includes native types for direct memory handling:

- **Hexadecimal:** `hex` (Stores base-16 numerical values directly).
- **Pointers:** `ptr`, `pointer`, or `address` (Stores memory addresses for raw data manipulation).

In strict mode, Wrench seamlessly allows assignments between memory types (`hex`, `ptr`) and standard `int` types without raising a semantic error, acknowledging that memory addresses are fundamentally integers at the hardware level.

3.3. Flexible Types (var & any)

For scenarios where dynamic typing or type inference is preferred:

- `var`: Instructs the compiler to infer the variable's type based on the assigned value during compilation. Once inferred, the type is statically locked.
- `any`: A universal wildcard type. It completely bypasses the Semantic Analyzer's strict type-checking, allowing a variable to hold or transition between any data type.

3.4. Variable Declaration

Variables are declared by specifying an optional access modifier, the type, the identifier, and an initial value. Every assignment must end with a semicolon (`;`).

```
int my_number = 42;
var inferred_text = "Wrench Compiler"; // Type becomes 'string'
hex mem_addr = 0x44000004;
```

Access modifiers (`public`, `private`) can be placed before the type. While primarily used for class fields, the parser allows them in standard variable assignments.

```
private int hidden_value = 100;
```

3.5. Type Checking & Casting

The Wrench compiler includes a strict Semantic Analyzer (`strict_mode`). When strict mode is enabled, the compiler enforces type safety with specific hardware-level tolerances:

Changed in version 0.1.1: Semantic analysis checks were expanded: * **Return Type Validation:** Verifies that function returns match declared signatures or fallback to `unit`. * **Condition Guards:** Enforces that conditional expressions (in `if`, `while`, `when`) evaluate to valid logical types rather than `unit`. * **Index Validation:** Confirms array access indices evaluate to `int` or `hex`. * **Class Hierarchy Scans:** Member lookup scans parent hierarchies via `get_field_type()`.

1. **Implicit Conversions:** Assigning an `int` to a `double` is natively permitted.
2. **Polymorphism (Upcasting):** Assigning a child class instance to a parent class type is strictly validated and allowed.
3. **Explicit Casting:** Developers can force a type conversion using the `cast` keyword.

```
double pi = 3; // Legal implicit conversion
var str_val = 42 cast str; // Explicit cast from int to string
```

Using `cast` translates directly into hardware-level data conversion instructions (e.g., `cvtsi2sd` for int to double, or dynamic allocations for int to string) within the Code Generator.

4. Control Flow

Wrench provides a familiar yet distinct set of control flow mechanisms, replacing some traditional keywords with more expressive alternatives.

4.1. Conditional Statements (if / butif / else)

Wrench uses `if` for initial conditions and introduces `butif` as the direct replacement for `else if` or `elif`. The fallback block is defined with `else`.

```
if (x > 10) {
    print("x is greater than 10;");
} butif (x == 10) {
    print("x is exactly 10;");
} else {
    print("x is less than 10;");
}
```

4.2. Switch / Case

Wrench supports `switch` statements for cleaner multi-condition branching.

Important: Unlike C or C++, Wrench does **not** feature implicit fall-through. The compiler automatically inserts jump instructions at the end of each `case` block. Therefore, you do not need to manually write `break;` at the end of a case.

```
switch (status_code) {
    case 200:
        print("Success;");
    case 404:
        print("Not Found;");
    default:
        print("Unknown Error;");
}
```

4.3. Loops (while & for)

While Loops: Execute as long as the condition evaluates to true.

```
while (count < 5) {  
    count += 1;  
}
```

For Loops & Limits: Wrench's `for` loop iterates over arrays or iterable objects using the `in` keyword. To loop through a range of numbers, Wrench provides the built-in `limits(start, end)` function, which generates an iterable sequence on the fly.

```
for i in limits(0, 10) {  
    print_int(i);  
}
```

4.4. Loop Control (break & goon)

Wrench uses standard loop control mechanisms but renames the `continue` keyword to `goon` (Go On) to better reflect its semantic purpose.

- `break;` Terminates the nearest enclosing loop entirely.
- `goon;` Skips the remaining code in the current iteration and forces the loop to proceed to the next iteration.

Both `break` and `goon` must be terminated with a semicolon.

```
for i in limits(0, 100) {  
    if (i == 50) {  
        break; // Stop the loop completely  
    } butif (i % 2 == 0) {  
        goon; // Skip even numbers  
    }  
    print_int(i);  
}
```

5. Memory & Arrays

Because Wrench does not rely on a standard C library (libc) or a Virtual Machine, its array and memory management systems interact directly with the operating system kernel.

5.1. Raw Pointer Arrays & The Hidden Header

In Wrench, arrays are **not** object instances (classes). They are contiguous blocks of raw memory accessed via pointer arithmetic.

To provide safety without the overhead of a class structure, the Wrench compiler implements a **Hidden Length Header**. When an array is allocated, the compiler requests extra memory from the kernel. It writes the length of the array into the first 8 bytes (64 bits), and then advances the pointer by 8 bytes before returning it to the program.

As a result, the pointer you interact with points directly to the first element, while the array's capacity remains safely hidden just behind the pointer (at `pointer - 8 bytes`).

5.2. Array Initialization & Bounds Checking

Arrays can be initialized using the array literal syntax `[]`.

```
var my_list = [10, 20, 30, 40];
print_int(my_list[2]); // Prints 30;
```

Out-of-Bounds Protection: Before any array read (`my_list[i]`) or write (`my_list[i] = x;`) operation, the compiler injects assembly instructions to read the hidden length header. If the requested index is less than 0 or greater than/equal to the array's length, the compiler throws a critical **Out of Bounds Error (Code: 99)** and routes the program to the exception handler to prevent memory corruption.

5.3. Strings as Character Arrays

Strings in Wrench are natively treated as arrays of characters. However, the compiler tracks the variable's evaluation type (`eval_type == 'char'`).

When you index a standard array (e.g., an array of integers), the compiler steps forward by 8 bytes per index. When you index a string, the compiler steps forward by exactly 1 byte.

```
var text = "Wrench";
text[0] = "F" cast char;
print(text); // Prints "French"
```

5.4. Manual Allocation and Deallocation

Wrench provides low-level functions to manually request and release raw memory pages from the kernel:

- ```alloc(size);``` Allocates a contiguous block of memory and returns a pointer. (Automatically applies the hidden length header logic).
- ```realloc(pointer, new_size);``` Resizes a previously allocated memory block.

- `delete pointer;` Because Wrench lacks a Garbage Collector, dynamically allocated arrays or objects must be manually freed using the `delete` keyword to prevent memory leaks.

```
var buffer = alloc(1024); // Allocate 1024 bytes
// ... use buffer ...
delete buffer;           // Free the memory back to the OS
```

6. Functions

Functions in Wrench are defined using the `define` keyword. The language supports standard named functions, anonymous functions (anfunc), and external foreign function interfaces.

6.1. Function Definition & Returns

A standard function requires a name, a typed parameter list, and a code block. Wrench does not require you to explicitly declare the return type of a function in the signature; it is inferred dynamically.

Added in version 0.1.1: Functions and methods support explicit return type annotations using the `-> ReturnType` syntax. If omitted, the return type defaults to `unit` or is inferred during semantic analysis.

To return a value from a function, use the `return` keyword followed by a semicolon. To return early without a value, simply use `return;`.

```
define calculate_area(int width, int height) {
    var area = width * height;
    return area;
}
```

6.2. Formatted Printing (printf)

Added in version 0.1.1: Wrench includes native `printf` format string processing. Format strings support interpolated variable names inside curly braces (e.g., `{name}`) as well as format specifier shortcuts.

Format specifier shortcuts include: `*`x` / ``X```: Format value as hexadecimal address or integer. `*`d` / ``i```: Format value as integer. `*`f``: Format value as double / float. `*`s``: Format value as string. `*`c``: Format value as character.

```
var x = 255;
printf("Value: {x:x}\n"); // Displays hex format 0xFF
```

6.3. Anonymous Functions (anfunc)

Wrench supports anonymous functions via the `anfunc` keyword. Anonymous functions do not have a name and are typically assigned to variables. This allows functions to be passed around dynamically as pointers.

```
var add_numbers = anfunc(int a, int b) {
    return a + b;
};

var result = add_numbers(15, 25); // Evaluates to 40;
```

6.4. External Functions (extern)

Because Wrench compiles directly to Assembly and conforms to standard binary interfaces, you can easily link it with external libraries (like C standard libraries or custom Assembly files).

Use the `extern` modifier before `define` to declare a function that exists outside the current codebase. External definitions do not have a code block and must end with a semicolon.

```
extern define printf(str format, var value) -> unit;
```

6.5. System V ABI & Calling Convention

At the hardware level, Wrench strictly adheres to the **System V AMD64 ABI** calling convention for x86-64 architectures.

When a function is called, the compiler automatically distributes the arguments according to this standard: 1. The first 6 arguments are loaded directly into CPU registers for maximum performance (`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`). 2. If a function receives more than 6 arguments, the remaining arguments are automatically pushed onto the stack before the function is called.

This native compliance ensures that Wrench can seamlessly interoperate with C libraries and OS kernel structures without requiring bridging layers or wrappers.

Note

Roadmap / Cross-Platform Support: Currently, the compiler's backend strictly targets the Linux System V ABI. Implementations for the Windows x64 ABI (which utilizes different parameter registers and requires a 32-byte shadow space) and MacOS architectures are scheduled for future minor releases.

7. Object-Oriented Programming (OOP)

Wrench provides a class-based Object-Oriented Programming (OOP) model. Unlike managed languages, Wrench's OOP operates directly on raw memory. Objects are essentially structured memory blocks, and methods are standard functions that receive a hidden pointer to the object instance.

7.1. Class Definition & Instantiation

Classes are defined using the `class` keyword. Inside a class, you can declare fields (variables) and methods (using the `define` keyword). The `init` method acts as the constructor and is called automatically when an object is created.

To instantiate an object, use the `new` keyword. This automatically allocates the exact amount of memory required for the object's fields via the kernel and invokes the `init` method.

```
class Player {
    int health;
    int level;

    define init(int initial_health) {
        this.health = initial_health;
        this.level = 1;
    }
}

var p1 = new Player(100);
```

7.2. Inheritance (extends)

Wrench supports single inheritance using the `extends` keyword. A child class inherits all fields and methods of its parent class, and the memory layout is sequentially expanded.

```
class Character {
    int health;
}

class Npc extends Character {
    int aggression_level;
}
```

7.3. Access Modifiers (public & private)

Fields and methods can be prefixed with access modifiers to control visibility. By default, members are `public`.

If a member is marked as `private`, the compiler strictly enforces access rules during the Semantic Analysis phase, preventing any external reads, writes, or method calls.

```
class Vault {  
    private int secret_code;  
  
    public define unlock() {  
        // Accessing private members internally is allowed  
        if (this.secret_code == 1234) {  
            print("Unlocked");  
        }  
    }  
}
```

7.4. The 'this' Keyword

Inside any class method, the `this` keyword is automatically available. It acts as a pointer to the current instance of the object. Under the hood (following the System V ABI), `this` is implicitly passed as the very first argument (`rdi` register) to the method.

7.5. Type Casting (Upcasting & Downcasting)

Wrench's strict semantic analyzer ensures object type safety but allows hierarchical type conversions using the `cast` keyword.

- **Upcasting:** Converting a child object to a parent type. This is implicitly allowed by the compiler but can also be done explicitly.
- **Downcasting:** Converting a parent reference back to a child type. This requires an explicit `cast`.

```
var main_player = new Player(100);  
main_player.level = 25;  
  
Character npc = main_player; // Implicit UPCAST  
var reverted_player = npc cast Player; // Explicit DOWNCAST  
  
print_int(reverted_player.level); // Prints 25;
```

8. Error Handling & Signals

Because Wrench lacks a Virtual Machine and background runtime, it implements error handling directly at the hardware and operating system level. It intercepts hardware faults (like Segmentation Faults) using OS-level signal handlers (`rt_sigaction`) and routes them to user-defined fallback blocks by restoring previous stack frames.

8.1. Attempt, Exclude, and Final

- ```attempt```: The block of code to be executed safely. Before entering, the compiler pushes the current CPU state (RSP, RBP, and the fallback address) to a linked list known as the Global Error Frame.
- ```exclude```: The fallback block executed if an error occurs. When a fault is intercepted, the compiler injects an implicit local variable named `err` (containing the error code) into this block's scope.
- ```final```: A block that is guaranteed to run regardless of whether an error occurred or not.

```
attempt {
    var arr = limits(0, 5);
    print_int(arr[10]); // Triggers an Out-of-Bounds memory fault
} exclude {
    print("Intercepted an error! Code: ");
    print_int(err); // Prints the hardware fault code (99)
} final {
    print("Execution completed;");
}
```

If an `exclude` block is omitted and an error occurs, the compiler executes the `final` block and automatically “bubbles up” the error to the next higher `attempt` frame.

8.2. Triggering Errors (trigger & allege)

Developers can manually invoke the error-handling mechanism using two keywords:

- ```trigger```: Immediately halts the current execution and throws a specific error code. The error code is caught by the nearest `exclude` block.
- ```allege```: Acts as an assertion. It evaluates a condition; if the condition is false, it automatically triggers a predefined assertion error (Error Code: 1).

```
allege x > 0; // Throws Error 1 if x is less than or equal to 0

if (status != 200) {
    trigger 404; // Throws a custom error code
}
```

8.3. Hardware-Level Fault Interception

In standard C or C++, accessing an invalid memory address (e.g., an array out-of-bounds or a null pointer) causes the OS to terminate the program immediately with a Segmentation Fault (SIGSEGV).

In Wrench, the compiler registers a custom signal handler (using Syscall 13) at startup. If a memory violation occurs (such as the Array Out of Bounds check failing and throwing code 99), the OS hands control over to Wrench's internal `sigsegv_handler`. The handler reads the Global Error Frame, unwinds the stack registers back to their safe state, and jumps directly into the user's `exclude` block without crashing the application.

9. Concurrency & Interrupts

Wrench avoids the complexity of user-space threading (like standard pthreads) and instead relies on true OS-level multi-processing. Concurrency is achieved natively by utilizing the kernel's process duplication mechanism (`fork`) and process waiting rules.

9.1. Asynchronous Execution (`async` & `wait/await`)

Functions can be marked with the `async` modifier. When an asynchronous function is called, the Wrench compiler invokes the kernel to fork a new child process. The child process executes the function's code in parallel, while the parent process immediately continues to the next line of code without blocking.

To pause the parent process and wait for a specific parallel task to finish, use the `wait` or `await` operators. These operators invoke the kernel (Syscall 61) to suspend the parent process until the child process completes its execution.

```
async define heavy_computation(int x) {  
    // ... intensive work ...  
}  
  
var process_id = heavy_computation(100);  
print("This prints immediately;");  
  
await process_id; // Parent pauses here until the computation is done  
print("This prints after the computation finishes;");
```

9.2. Synchronization Units (`sync unit` & `with`)

Wrench provides a unique mechanism for grouping code blocks that need to run concurrently or sequentially, known as Synchronization Units.

- ```sync unit```: Defines a named block of code. It does not execute immediately; it merely registers the code block.
- ```with```: Executes a predefined unit. By default, calling `with` runs the unit asynchronously (forking a new process). If you use `wait with`, the unit executes synchronously in the current process.

```
sync unit download_data {
    // ... network operations ...
}

with download_data {
    print("Data download started in the background;");
}

wait with download_data {
    print("This runs synchronously after the unit completes;");
}
```

9.3. Event-Driven Observers (when & global when)

Wrench introduces an interrupt-like polling architecture using the `when` keyword.

A `when` block acts as a background observer. Once declared, the compiler forks a lightweight child process that immediately enters a sleep state (using `nanosleep`). It wakes up every 10 milliseconds to evaluate its condition. If the condition becomes true, the process executes the internal block and then terminates safely.

- ```when```: Observes variables within the current local scope.
- ```global when```: Escalates the observer to the highest hierarchy, allowing it to continuously monitor global application state regardless of the current local scope.

```
var ready = false;

when (ready ==? true) {
    print("The system is now ready!");
}

// ... other code ...
ready = true; // The background observer wakes up, detects the change, and prints.
```

10. Modules & Linker

Wrench supports modular programming by allowing you to split your code into multiple `.wr` files. The compiler's frontend resolves all imports, merges the Abstract Syntax Trees (AST) prior to semantic analysis, and handles external object linking natively.

10.1. Importing Modules (include)

To import an entire Wrench module, use the `include` keyword followed by the file name (as a string). The compiler automatically appends the `.wr` extension if it is omitted.

If a module name starts with `std/`, the compiler looks for it in the standard library directory. Otherwise, it searches relative to the current working directory.

```
include "math_utils"; // Imports math_utils.wr from the current directory
include "std/sys";    // Imports sys.wr from the standard library
```

10.2. Selective Imports & Aliasing (from ... include ... as)

If you only need a specific function, class, or variable from another module, use the `from ... include` syntax. This prevents the entire module from polluting your global scope.

You can also rename the imported item locally using the `as` keyword to prevent naming collisions.

```
from "math_utils" include calculate_area;

from "std/crypto" include hash_password as bcrypt_hash;
var secure_pass = bcrypt_hash("my_secret");
```

10.3. Linking External Object Files (link)

Wrench is designed to interoperate directly with compiled binaries from other languages (like C, C++, or raw Assembly). You can instruct the Wrench compiler to pass specific object files (`.o`) to the GNU Linker (`ld`) during the final compilation stage using the `link` keyword.

```
link "external_physics_engine.o";
extern define apply_gravity(ptr object); // Declared in the .o file
```

When the Wrench CLI compiles this file, it gathers all `link` directives and appends them to the final linker command, ensuring the external machine code is seamlessly merged into the final executable.

Wrench Standard Library

The Wrench Standard Library provides essential built-in modules for data structures, system operations, file I/O, process execution, pattern matching, mathematical calculations, and low-level memory utilities.

Changed in version 0.1.1: All standard library functions and methods have been updated with explicit return type annotations (`-> ReturnType`). Internal embedded test suites were stripped from all library source files to optimize compilation size.

1. Collections (`collections.wr`)

The `collections.wr` module provides dynamic data structures: `List` and `Map`. Since Wrench does not have a Garbage Collector, both structures require explicit memory cleanup using their respective `destroy()` methods.

1.1. List

A dynamically resizing array structure.

- **init():** Initializes the list and allocates raw memory.
- **push(var item):** Adds an element to the end of the list.
- **get(int index) -> any:** Returns the item at the specified index. Throws Out Of Bounds Error (Code 99) if invalid.
- **set(int index, var item):** Updates the value at the specified index.
- **pop() -> any:** Removes and returns the last item in the list.
- **clear():** Deletes the underlying memory and resets the list state.
- **destroy():** Frees the raw memory allocated for the list's data.

```
include "std/collections.wr";

var my_list = new List();
my_list.push(10);
my_list.push(20);
print_int(my_list.get(1) cast int); // Prints 20

my_list.destroy();
delete my_list;
```

1.2. Map

A key-value dictionary structure operating on dual parallel lists.

- **init():** Initializes internal storage lists.
- **set(str key, var value) -> int:** Adds or updates a key-value pair.

- `get(str key) -> any`: Returns the value associated with the key.
- `has(str key) -> bool`: Returns `true` if the key exists in the map, otherwise `false`.
- `destroy()`: Frees internal key and value lists.

2. Error Management (error.wr)

Provides a hierarchical exception and error handling system built upon `BaseException` and branching into `Exception`, `FatalError`, and `Warning`.

2.1. The Root Class

- **BaseException**: Base error class initialized with a `message` and `code`. Provides `print_err()`.

2.2. Main Branch 1: Exception

Standard run-time errors including `ArithmeticError`, `LookupError` (`IndexError`, `KeyError`), `NameError`, `ValueError`, `SyntaxError`, `OSError` (`FileNotFoundError`, `PermissionError`, `FileExistsError`), `TypeError`, and `RuntimeError`.

2.3. Main Branch 2: FatalError

Critical system failures carrying a `fault_addr` hex property, including `SystemExit`, `KeyboardInterrupt`, `GeneratorExit`, `MemoryError`, `SystemError`, and `RecursionError`.

2.4. Main Branch 3: Warning

Non-fatal alerts including `DeprecationWarning`, `UserWarning`, `SyntaxWarning`, and `RuntimeWarning`.

3. File System (fs.wr)

Handles file and directory operations using direct OS kernel calls.

3.1. The File Class

- `init(str file_path)`: Opens file in read-only mode, measures size via `sys_lseek`, and loads contents into memory.
- `read() -> str`: Returns loaded file content.
- `close()`: Closes file descriptor and frees the allocated buffer.

3.2. File Operations

- `write_file(str file_path, str content) -> bool`: Overwrites file with content.

- `append_file(str file_path, str content) -> bool`: Appends content to file end.
- `delete_file(str file_path) -> bool`: Unlinks file from filesystem.

3.3. Directory Operations

- `dir_list(str dir_path) -> List`: Reads directory structure using kernel dirent syscalls and returns a `List` of filenames.

4. Math (`math.wr`)

Provides mathematical constants, trigonometry, logarithms, rounding, and PRNG.

4.1. Core Functions

- `rad_to_deg(double rad) -> double` / `deg_to_rad(double deg) -> double`: Angle conversion.
- `sin(double x) -> double` / `cos(double x) -> double` / `tan(double x) -> double`: Trigonometry.
- `asin(double x) -> double` / `acos(double x) -> double` / `atan2(double y, double x) -> double`: Inverse trigonometry.
- `pow(double base, double exp) -> double` / `sqrt(double x) -> double` / `cbrt(double x) -> double`: Powers and roots.
- `log(double x) -> double` / `log10(double x) -> double` / `log2(double x) -> double`: Logarithms.
- `num_abs(var x) -> any` / `fabs(double x) -> double`: Absolute values.
- `floor(double x) -> double` / `ceil(double x) -> double` / `round(double x) -> double`: Rounding.
- `rand() -> int` / `rand_range(int min_val, int max_val) -> int` / `rand_float() -> double`: Pseudo-random generation.
- `dist2d(double x1, double y1, double x2, double y2) -> double`: 2D Euclidean distance.

5. String (`string.wr`)

Provides string manipulation routines.

- `str_len(str text) -> int`: Returns string character count.
- `substring(str text, int start, int end) -> str`: Returns extracted substring.
- `uppercase(str text) -> str` / `lowercase(str text) -> str`: ASCII casing conversions.
- `split(str text, char delimiter) -> var`: Splits string into array by delimiter.
- `parse_int(str text) -> int`: Converts string to integer.
- `to_string(int number) -> str`: Converts integer to string.
- `concat(str left, str right) -> str`: Concatenates two strings.

6. System (`sys.wr`)

Changed in version 0.1.1: Restructured around a centralized `SysModule` instance named `sys`.

- **sys.argv**: List of CLI arguments.
- **sys.platform**: String identifier for running platform (e.g., `"linux"`).
- **sys.exit(int code)**: Terminates execution with given status code.
- **sys.getsizeof(var obj) -> int**: Calculates allocated byte size of an object or memory buffer.
- **args()** -> **List**: Legacy wrapper returning CLI argument list.
- **execute(str program, List cmd_args) -> int**: Forks and executes an external binary.

```
include "std/sys.wr";

print(sys.argv.get(0) cast str);
if (sys.argv.length < 2) {
    sys.exit(1);
}
```

7. Time (time.wr)

- **now()** -> **int**: Returns UNIX epoch time in seconds.
- **now_ms()** -> **int**: Returns epoch time in milliseconds.
- **ticks()** -> **int**: Returns monotonic clock milliseconds.
- **sleep(int ms) -> int**: Pauses process execution for specified duration.
- **measure(var func_ptr) -> int**: Profiles execution time of target function pointer in milliseconds.
- **epoch_to_utc(int epoch) -> var**: Converts epoch to `[year, month, day, hr, min, sec]` array.
- **now_utc()** -> **var**: Returns current UTC date/time array.

8. Argument Parser (argparser.wr)

Added in version 0.1.1: Provides CLI command line argument definition and sub-command parsing.

8.1. Argument & ArgumentParser Classes

- **ArgumentParser.init(str desc)**: Creates parser instance.
- **add_argument(str name, str arg_type, str action)**: Defines expected flag or positional parameter (actions: `"store"`, `"store_true"`, `"count"`).
- **add_subparsers(str command_name, str desc) -> ArgumentParser**: Adds sub-command parser.
- **parse_args(List sys_args) -> Map**: Parses argument list and returns a `Map` containing options.
- **destroy()**: Frees internal arguments and subparser structures.

```
include "std/argparser.wr";

var parser = new ArgumentParser("Wrench CLI Tool");
parser.add_argument("-v", "bool", "store_true");
parser.add_argument("-o", "str", "store");

var opts = parser.parse_args(sys.argv);
```

9. Memory Cloning (clone.wr)

Added in version 0.1.1: Provides raw memory duplication, filling, and array element swapping.

9.1. CloneModule Class (clone)

- **clone.clone_mem(var dest, var src, int count):** Copies `count` bytes from source to destination address.
- **clone.fill(var target, int value, int count):** Sets target memory buffer bytes to `value`.
- **clone.swap_arr(var arr, int i, int j):** Swaps elements at indices `i` and `j`.
- **clone.clone(var obj) -> var:** Reads object size via hidden header and returns a newly allocated duplicate clone in heap memory.

10. Operating System (os.wr)

Added in version 0.1.1: Provides system path management, current directory querying, environment inspection, and OS error mapping.

10.1. PathModule Class (os.path)

- **os.path.join(str p1, str p2) -> str:** Joins path components with directory separators.
- **os.path.dirname(str p) -> str:** Extracts directory component of a path.
- **os.path.normpath(str p) -> str:** Resolves `.` and `..` relative path components.
- **os.path.exists(str p) -> bool:** Checks if path exists on file system via kernel access check.
- **os.path.abspath(str p) -> str:** Resolves path into an absolute file system path.

10.2. OSModule Class (os)

- **os.getcwd() -> str:** Queries current working directory path via Syscall 79 (`getcwd`).
- **os.remove(str p) -> bool:** Unlinks file path.
- **os.rmdir(str p) -> bool:** Removes directory path.
- **os.makedirs(str p, bool exist_ok):** Recursively creates path directory tree.
- **os.access(str p, int mode) -> bool:** Verifies file permission access flags.
- **os.getenv(str key) -> str:** Searches process environment variables for key value.

```
include "std/os.wr";

var cwd = os.getcwd();
var config_path = os.path.join(cwd, "config.json");

if (os.path.exists(config_path)) {
    var user = os.getenv("USER");
}
```

11. Regular Expressions (regex.wr)

Added in version 0.1.1: Provides regular expression pattern searching, matching, and text substitution.

- **compile(str pattern)** -> **RegexPattern**: Compiles regular expression pattern string.
- **match(str pattern, str text)** -> **str**: Tests match at string head.
- **finditer(str pattern, str text)** -> **List**: Scans text and returns a **List** of matching substrings.
- **split(str pattern, str text)** -> **List**: Splits text into a **List** using pattern matches as delimiters.
- **sub(str pattern, str repl, str text)** -> **str**: Replaces matches in text with replacement string.

12. Subprocess Management (subprocess.wr)

Added in version 0.1.1: Provides child process spawning, process I/O pipe redirection, signal handling, and execution waiting.

12.1. Classes and Global Functions

- **CompletedProcess**: Process execution snapshot containing **cmd**, **returncode**, **stdout**, **stderr**, and merged **output**.
- **Popen.poll()** -> **int**: Non-blocking status check on child process PID.
- **Popen.wait_proc(int timeout_ms)** -> **int**: Waits for process completion or sends **SIGKILL** (-9) if timeout expires.
- **Popen.communicate()** -> **CompletedProcess**: Reads output pipes, closes file descriptors, waits for termination, and returns process summary.
- **run(var cmd_array, bool capture_output, int timeout_ms)** -> **CompletedProcess**: Spawns process using kernel **fork** (Syscall 57) and **execve** (Syscall 59), handling pipe creation (Syscall 22) and duplication (Syscall 33).

```
include "std/subprocess.wr";

var cmd = ["/bin/ls", "-la"];
CompletedProcess proc = run(cmd, true, 5000);

if (proc.returncode ==? 0) {
    print(proc.stdout);
}
```

Compiler Internals

This section provides a deep dive into the internal architecture of the Wrench compiler. It is intended for core developers, contributors, and those curious about how Wrench translates raw text into bare-metal machine code.

1. The Compilation Pipeline

The Wrench compiler operates as a multi-pass pipeline, orchestrated primarily by the Command Line Interface (CLI) module (`wrench.py`). The lifecycle of a Wrench program involves the following stages:

- Lexical Analysis (Frontend):** The source file (`.wr`) is read as a raw string and converted into a stream of tokens.
- Parsing (Frontend):** The token stream is processed by a Recursive Descent Parser to construct an Abstract Syntax Tree (AST).
- Import Resolution (Middle-end):** The compiler scans the AST for `include` and `link` nodes. If found, it recursively reads, tokenizes, and parses the imported modules, merging their ASTs into the main tree to form a unified, flattened AST.
- Semantic Analysis (Middle-end):** A Visitor walks the unified AST to enforce scoping rules, validate types (Strict Mode), and annotate nodes with hardware-level evaluation types.
- Code Generation (Backend):** The annotated AST is compiled down to x86-64 System V ABI Assembly code (`.asm`).
- Assembling & Linking (Toolchain):** The compiler invokes `nasm` to convert the Assembly into an ELF64 object file (`.o`), and then calls the GNU Linker (`ld`) to link it with any external objects, producing the final standalone executable.

Changed in version 0.1.1: The compilation pipeline CLI status reporting was standardized, and `CodeGen` now directly ingests the `SemanticAnalyzer` state to perform synchronized type lowering and return-type verification.

2. Lexical Analysis

The Lexer converts the raw source string into a list of structured tokens. Wrench uses a sequential Regular Expression (regex) pattern matcher to achieve this.

Token Matching Order: The order of regex rules is critical to prevent partial matches. For example, multi-line comments (`/# ... #/`) are evaluated before single-line comments (`//`), and double-character operators (like `=?` or `+=`) are matched before single-character operators.

Added in version 0.1.1: Added the `ARROW` token (`->`) to the token specification for explicit return type signatures. All internal lexer diagnostic messages were converted to English.

Token Structure: Whenever a valid substring is matched, the Lexer generates a tuple containing four elements: `(kind, value, line_num, column)`

Tracking the `line_num` and `column` directly at the lexical level allows the Parser and Semantic Analyzer to throw highly precise, pinpoint error messages when syntax or typing violations occur.

Keyword Mapping: To keep the regex engine fast, Wrench does not use separate regex rules for every keyword. Instead, any continuous string of letters is initially categorized as an `IDENTIFIER`. Before appending the token, the Lexer checks if the identifier exists within the predefined `KEYWORDS` set. If a match is found, the token's kind is immediately mutated from `IDENTIFIER` to `KEYWORD`.

3. Abstract Syntax Tree & Parsing

The Parser transforms the flat token stream into a hierarchical Abstract Syntax Tree (AST). It is implemented as a **Recursive Descent Parser**, predicting the structural grammar by reading tokens sequentially via `advance()` and `peek()` methods.

Added in version 0.1.1: Introduced the `expect_identifier()` helper method. This guard prevents language keywords or members of the `BUILTIN_FUNCTION` set (e.g., `print`, `alloc`, `len`) from being redefined or misused as identifier names. Function and method parsers were extended to parse arrow return signatures (`-> ReturnType`).

Order of Operations (Operator Precedence): Mathematical and logical precedence is strictly enforced by the call stack hierarchy within the parser. The evaluation flows from the lowest precedence (broadest expressions) down to the highest precedence (tightest binding): 1.

`comp_expr()` : Logical operations (`and`, `or`, `not`). 2. `rel_expr()` : Relational comparisons (`>?`, `<=`). 3. `expr()` : Addition and subtraction (`+`, `-`). 4. `term()` : Multiplication, division, modulo, and power (`*`, `/`, `%`, `^`). 5. `factor()` : Unary operations and type casting (`cast`). 6. `base_factor()` : Literals, variable access, function calls, and array/member indexing.

Node Generation: Every matched grammar rule produces a specific Node object (e.g., `BinOpNode`, `IfNode`, `AttemptNode`). The parser also enforces the strict semicolon rule at the statement level, halting compilation immediately with precise line/column data if a terminator is missing.

4. Semantic Analysis & Type Checking

Before any machine code is generated, the AST must be validated and annotated. The Semantic Analyzer acts as the crucial middle-end, traversing the AST using the **Visitor Design Pattern**.

Changed in version 0.1.1: Semantic Analysis was significantly expanded: * **Return Type**

Tracking: Tracks target return types per scope using `self.current_return_type` to ensure

`return` expressions match declared function signatures. * **Built-in Type Definitions:**

Maintains `BUILTIN_FUNCTION_TYPES` mapping expected return types for native functions (e.g.,

`len` -> `int`, `read` -> `string`, `alloc` -> `ptr`). * **Validation Helpers:** Added

`check_type_compatibility()`, `check_boolean_condition()`, and `check_index_type()` to catch type

mismatches before code generation. * **Terminology:** Standardized non-value evaluation types

to `unit` (replacing `void`).

Scope & Environment Tracking: The analyzer maintains a stack of dictionaries

(`self.environments`) to track variable scope. * Entering a block (e.g., a loop or function) pushes a new dictionary onto the stack. * Exiting the block pops the dictionary off. This architecture natively supports variable shadowing and guarantees that local variables do not leak into outer scopes.

Type Tagging for Code Generation: The Code Generator relies on knowing the exact data type to select the correct hardware instruction (e.g., integer ALU operations vs. FPU/XMM floating-point operations). The Semantic Analyzer evaluates the tree and attaches `eval_type` and `operand_type` attributes to the AST nodes dynamically.

Strict Mode & Tolerances: When `strict_mode` is enabled, the analyzer intercepts invalid type assignments. However, to support low-level system programming, it explicitly allows certain implicit conversions: * `int` to `double` . * Standard numerical types to hardware pointer types (`hex` , `ptr`). * Implicit Upcasting (verifying class inheritance hierarchies via an internal `class_hierarchy` map).

5. Code Generation & System V ABI

The Code Generator (CodeGen) is the most complex backend component of the Wrench compiler. It operates as an AST Visitor that translates semantic nodes directly into raw x86-64 NASM Assembly strings.

Rather than relying on an intermediate representation (IR) like LLVM, Wrench generates assembly natively, embedding OS-specific syscalls and hardware instructions into the final output.

5.1. Dedicated Assembly Buffers (*functions_code*)

Added in version 0.1.1: Free functions, class methods, and anonymous functions (`anfuncs`) are no longer emitted inline into the main execution pipeline with manual jump-over labels (`jmp AFTER_...`). Instead, CodeGen uses a dedicated assembly buffer (`self.functions_code`). Function bodies are collected independently and emitted before `_start` , un-cluttering the main execution flow and unifying exit labels (`exit_func_{name}`).

5.2. Condition Branching & Inverse Jumps (*generate_branch*)

Added in version 0.1.1: Condition evaluation in `if` and `while` structures was optimized via `generate_branch()` . Rather than setting boolean registers and testing them (`sete` / `movzx` / `test`), CodeGen evaluates binary comparison operands directly and emits direct inverse jump instructions (e.g., `jne` , `jle` , `jae`) to jump straight to target labels.

5.3. Stack Memory, Alignment, and Register Optimizations

- **16-Byte Stack Alignment:** Stack frame allocation in methods and functions respects 16-byte System V ABI alignment requirements.
- **32-Bit Register Footprint Reduction:** Resetting registers or passing small constants uses 32-bit register operations (e.g., `xor eax, eax` instead of `mov rax, 0` , `mov edi, 1` instead of `mov rdi, 1`), saving binary code bytes.
- **Formatted Output Engine:** `printf` format strings are parsed at compile time via `_parse_format_string()` into static text and dynamic placeholders, building target AST nodes and invoking specialized assembly printing routines (`print_int` , `print_float` , `print_hex` , `print_string`).

5.4. Hardware-Level Exception Interception

Wrench's `attempt` / `exclude` exception handling operates entirely via OS signal trapping and stack unwinding.

1. **The Global Error Frame:** Wrench maintains a pointer in the `.data` section (`global_err_frame`) that points to a linked list of active `attempt` blocks.
2. **State Saving:** Before entering an `attempt` block, CodeGen pushes the current `rsp` , `rbp` , and the assembly label address of the `exclude` block onto the stack, registering it to the global frame.
3. **Signal Handler (SIGSEGV):** The Wrench executable header includes an `rt_sigaction` (Syscall 13) call that intercepts hardware crashes (like Segmentation Faults). If the program attempts to access invalid memory (or an Out-of-Bounds check fails), the OS executes Wrench's internal `sigsegv_handler` .
4. **Unwinding:** The handler reads the `global_err_frame` , restores the CPU registers (`rsp` , `rbp`) to their safe state, and executes a `jmp` instruction directly into the `exclude` block, entirely bypassing a fatal crash.

5.5. Multi-Processing (Forking) over Threading

Wrench does not implement POSIX threads (pthreads). Instead, concurrency is handled through true multi-processing.

- **Async Execution:** The `async` modifier translates directly to a `fork` syscall. The parent process continues execution, while the child executes the isolated function.
- **Polling (The 'when' keyword):** Wrench's observer pattern forks a lightweight background process. CodeGen implements a continuous loop that invokes Syscall 35 (`nanosleep`) to sleep for 10 milliseconds, waking up only to evaluate the condition before sleeping again.

Roadmap & Future Vision

Wrench is currently in its **Prototype Phase**, written in Python and strictly targeting Linux x86-64 via System V ABI. This roadmap outlines the strategic phases to evolve Wrench from a Python-based prototype into a fully bootstrapped, self-hosting, cross-platform systems programming language.

Phase 1: Prototype Completion & Syntax Freeze (v0.1.x)

The primary goal of Phase 1 is to finalize the core language specifications and standard library foundations before moving away from the Python environment.

- **Essential Library Integration:** Finalize and integrate critical standard library modules (such as `regex.wr` and advanced Error/IO libraries).
- **Syntax & Semantics Freeze:** Lock down the language grammar and strict mode typing rules.
- **Backward Compatibility Baseline:** Establish a strict versioning transition plan to ensure that code written for the frozen v0.1.x syntax can be compiled by future bootstrapped versions without breaking.
- **Raw Syscall Foundation:** Solidify the native Linux syscall bridges for file I/O, networking, and memory allocation without relying on external C libraries.

Phase 2: Bootstrapping (v0.2.0)

Phase 2 begins the process of rewriting the compiler in Wrench itself.

- **Native Rewrites:** Port the Python modules (`lexer.py`, `parser.py`, `semantic.py`, `codegen.py`) directly into Wrench (`lexer.wr`, `parser.wr`, etc.).
- **Data Structure Migration:** Replace Python's dynamic data structures (lists, dictionaries) and dynamic memory model with Wrench's static arrays, hidden length headers, and custom AST node classes.
- **Recursion & Memory Limits:** Implement robust stack depth and recursion management in the native parser to handle AST generation without the safety net of Python's memory manager.

Phase 3: True Self-Hosting (v1.0.0)

This phase proves the language's independence. Wrench will compile its own compiler.

- **Stage 1 (Cross-compiling the compiler):** Use the Python prototype to compile `wrench.wr`, producing the first native executable: `wrench-v1`.
- **Stage 2 (Self-compiling):** Use `wrench-v1` to compile `wrench.wr` again, producing `wrench-v2`.
- **Verification & Diffing:** Implement a binary diffing/hashing algorithm to mathematically prove that the output of `wrench-v1` and `wrench-v2` are completely identical, confirming that the native compiler's logic is flawlessly self-sustaining.

Phase 4: Cross-Platform Expansion (v1.1.0+)

Expanding Wrench beyond Linux x86-64 by introducing target triples and dynamic OS bridges.

- **Target Triple Architecture:** Implement the `--target` CLI flag to allow cross-compilation (e.g., compiling Windows binaries from a Linux host).
- **Windows x64 Support:** * Adapt CodeGen to output the Windows ABI (implementing the 32-byte shadow space and Windows register calling conventions). * Build a native OS bridge interacting with the Win32 API and `kernel32.dll`. * Integrate compatible linker alternatives to replace GNU Linker (`ld`).
- **macOS (Apple Silicon / ARM64 & x86):** * Adapt NASM generation rules for ARM64 architectures. * Build a modular POSIX and `libSystem` core bridge specifically for the macOS target.

Phase 5: Optimizations & Ecosystem Tooling (v1.x.x)

Enhancing developer experience, performance, and CI/CD pipelines.

- **Compiler Optimization Passes:** * Implement Dead Code Elimination during the Semantic Analysis phase. * Advance the Register Allocation algorithms to reduce reliance on stack memory (optimizing beyond the standard 6-register System V rule).
- **Developer Tooling:** * Build a native Package Manager and a standardized Code Formatter (Linter). * Develop a VSCode Extension (VSIX) that communicates directly with the Wrench compiler for real-time error highlighting and AST analysis.
- **CI/CD & Virtualized Testing:** * Setup comprehensive GitHub Actions matrices. * Integrate local cross-platform verification using QEMU and Wine to map and intercept hardware virtualization error codes automatically.

Phase 6: Advanced Memory & OS Level Architecture

The long-term vision pushes Wrench into extreme low-level environments.

- **Memory Safety & Leak Analysis:** Develop advanced static analysis tools to prevent memory leaks when relying on manual `alloc` and `delete`.
- **Optional Garbage Collection:** Introduce an *optional* GC module that can be toggled on for application development, or completely detached for kernel-level programming.
- **The Ultimate Goal - Wrench OS:** Utilize Wrench to write a custom Operating System Kernel, beginning with defining the Interrupt Descriptor Table (IDT) and handling hardware CPU interrupts natively in Wrench.

Wrench v0.1.1 Release Notes

Version 0.1.1 is a feature and cleanup release focusing on semantic analyzer integration with code generation, compiler backend hardening, parser/lexer expansions, and a newly added suite of standard library modules.

New Standard Library Modules

- `std/argparser.wr`: Argument parsing library supporting `Argument`, `ArgumentParser`, flags, actions, and subparsers.
- `std/clone.wr`: Memory cloning, array cell swapping, target memory filling, and dynamic `clone()` helpers.
- `std/os.wr`: Path operations (`join`, `dirname`, `normpath`, `abspath`) and OS subsystem helpers (`getcwd`, `remove`, `rmdir`, `makedirs`, `access`, `getenv`) with OS error mapping].
- `std/regex.wr`: Regular expression matching utilities including `compile`, `match`, `finditer`, `split`, and `sub`.
- `std/subprocess.wr`: Process management and IPC library utilizing native x86-64 Linux syscalls (`SYS_FORK`, `SYS_EXECVE`, `SYS_PIPE`, `SYS_DUP2`, `SYS_WAIT4`, `SYS_KILL`) with `Popen` and `CompletedProcess` abstractions].

Compiler Backend & Code Generation (codegen.py)

- **Semantic Integration:** Synchronized `CodeGen` with `SemanticAnalyzer` to pass return-type metadata, strict variable types, and class member structures.
- **Isolated Function Section:** Decoupled free functions, methods, and anonymous functions into a dedicated `functions_code` assembly block linked into the executable entry point.
- **Conditional & Formatting Optimizations:**
 - Implemented `generate_branch` to convert boolean conditions directly into inverse jump assembly instructions.
 - Added format string parsing and placeholder AST construction for native `printf` support.
- **Assembly Routine Enhancements:**
 - Added and updated runtime assembly routines: `print_string` (null checking), `print_hex`, `concat_strings`, `compare_strings`, `alloc`, `get_mem32`, and `get_len`.
 - Replaced 64-bit register moves with 32-bit register alternatives for stack space optimization,

- Unified function exit labels and strengthened array bounds checking with SIGSEGV stack recovery.

Parser & Lexer Improvements (parser.py / lexer.py)

- **Explicit Return Types:** Added function and method return-type syntax support via the `->` (ARROW) token.
- **Identifier Validation:** Added `expect_identifier` helper and `BUILTIN_FUNCTION` protections to prevent keywords or built-in routines from being redefined as identifiers.
- **Error Diagnostics:** Added token context window logging for syntax parse errors.

Semantic Analyzer (semantic.py)

- **Type Tracking & Hierarchy:** Expanded semantic checks to track function signatures, class methods, class fields, and inheritance hierarchies.
- **Built-in Type Map:** Added `BUILTIN_FUNCTION_TYPES` mapping to enforce proper call/return evaluation for language built-ins.
- **Evaluation Tagging:** Extended visitor rules across AST nodes (Blocks, Conditions, Loops, Returns, Assignments, Class Definitions, Index Access) to infer and validate `eval_type`.

CLI & Pipeline (wrench.py)

- Updated execution workflow ordering: Tokenize -> Parse -> Resolve Imports -> Semantic Analysis -> CodeGen.
- Improved import alias resolution to preserve line and column metadata.
- Added traceback logging on compilation exceptions.

Standard Library Refactoring & I18n

- **Explicit Type Signatures:** Annotated standard library functions with return types (`->`) across `collections`, `string`, `fs`, `math`, `time`, and `sys`.
- **SysModule Migration:** Restructured `std/sys.wr` into a structured `SysModule` object.
- **Cleanup & Translation:** Removed embedded inline test harnesses across standard library files and translated Turkish error messages/comments into English.